

# Vector Databases for Semantic Search

## Content

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>I. Introduction</b>                              | <b>2</b>  |
| A. Motivation                                       | 2         |
| B. Context                                          | 2         |
| C. Research Purpose                                 | 2         |
| <b>II. Vector Databases: Definition and Purpose</b> | <b>3</b>  |
| A. What is a vector database?                       | 3         |
| B. Why do we need vector databases?                 | 3         |
| C. Where do we use vector databases?                | 3         |
| <b>III. Embeddings and Vector Representations</b>   | <b>4</b>  |
| A. What embeddings are?                             | 4         |
| B. How do embeddings work?                          | 4         |
| C. The role of embeddings in Semantic Search        | 5         |
| <b>IV. From Keyword Search to Semantic Search</b>   | <b>5</b>  |
| A. Keyword Search Limitations                       | 5         |
| B. Semantic Search Concept                          | 5         |
| C. Benefits / Transition                            | 6         |
| <b>V. Indexing Techniques</b>                       | <b>6</b>  |
| A. Purpose                                          | 6         |
| B. Methods                                          | 7         |
| C. Advanced Optimizations                           | 13        |
| <b>VI. Modern Vector Database Systems</b>           | <b>13</b> |
| A. About Modern Vector Database Systems             | 13        |
| B. Architecture / Features                          | 14        |
| C. Examples / Applications                          | 14        |
| <b>VII. Advantages and Limitations</b>              | <b>16</b> |
| A. Advantages                                       | 16        |
| B. Limitations                                      | 17        |
| C. Practical Considerations                         | 17        |
| <b>VIII. Recent Trends and Future Directions</b>    | <b>18</b> |
| A. Trends                                           | 18        |
| B. Integration with AI / LLMs                       | 21        |
| C. Future Research / Opportunities                  | 22        |
| <b>IX. Conclusion</b>                               | <b>23</b> |
| <b>References</b>                                   | <b>24</b> |

# **I. Introduction**

## **A. Motivation**

The topic of vector databases for semantic search caught my interest because it highlights a transformative idea: complex meanings, similarities, and recommendations can be represented and processed as numbers. The way embeddings convert text into vectors, and how databases use these vectors to quickly find related information, feels very new and powerful. This numerical approach is used in many everyday AI applications, and understanding the technologies behind it offers insight into how modern intelligent systems operate.

## **B. Context**

Nowadays applications generate huge amounts of unstructured data (text, images or even audio features included). Because of that, there are demanded specialised systems capable of efficiently supporting this type of data.

The traditional keyword search cannot capture relationships between concepts or different contexts, while AI models can, but only as long as their embeddings are efficiently stored and searchable in an easy way. This is why vector databases appeared: to support high-dimensional vector storage, fast similarity search and easy scalable semantic search, while using algorithms and special indexing techniques to handle huge volumes of data. This capability to represent complex information numerically enables systems to identify in an efficient way certain similarities, to support recommendations and to interpret user intent in ways that traditional keyword-based search cannot achieve.

## **C. Research Purpose**

From search engines to AI assistants and recommendation systems, vector databases are increasingly shaping modern applications nowadays. This report explains the transition from keyword-based to semantic search, introduces embeddings and vector representations and provides an overview of vector databases, their indexing techniques, the modern systems and current trends that are used or in development, along with their advantages and limitations.

Newer methods aim to improve vector search by moving beyond simple methods, using techniques that capture meaning better and reduce redundant results. These methods are studied in this report too.

## II. Vector Databases: Definition and Purpose

### A. What is a vector database?

A vector database is a specialized database system designed to store, index and query high-dimensional embeddings generated by modern AI applications, such as large language models (LLMs). It enables efficient similarity search and easy semantic retrieval, while handling challenges that traditional keyword-based or relational databases cannot manage. *(Pan, Wang & Li, 2023)*

### B. Why do we need vector databases?

First of all, vector databases are needed for handling large-scale unstructured data, such as text, audio and images, because modern applications work a lot on these types of data and traditional databases and search methods cannot efficiently manage or query them. *(Pan, Wang & Li, 2023 - section 1)*

Because of the large-scale nature of the unstructured data, vector databases are also needed for high-dimensional vector storage (embeddings) with fast retrieval, considering that searching through embeddings in an efficient way requires specialized indexing and retrieval mechanisms, which traditional relational databases cannot handle at scale. *(Pan, Wang & Li, 2023 - sections 2 & 3)*

Moreover, vector databases are needed for their semantic search capability. Keyword search cannot capture relationships between concepts or context, but vector databases allow storing embeddings to enable semantic similarity search, so systems can find related items based on meaning and recommendations, and not only on exact words. *(Pan, Wang & Li, 2023 - section 1)*

Vector databases have applications in real-world systems. They power modern search engines, recommendation systems, AI assistants based on LLMs and scientific data analysis. Without them, semantic search and recommendation at scale would be very impractical. *(Alfredo, 2025 - section 1)*

### C. Where do we use vector databases?

Vector databases are used in a variety of applications that require efficient and scalable semantic search, including the modern ways for information retrieval, where semantic similarity replaces the exact keyword matching.

For example, they are demanded in e-commerce applications and in different recommendation platforms, because data can be efficiently managed and searched based on similarity. *(Pan, Wang & Li, 2023 - section 1; Raja & Vats, 2025 - section 1)* Also, vector databases are used in fields such as astronomy, climate research and even genomics, in scientific and large-scale data applications. *(Alfredo, 2025 - section 1)*

### III. Embeddings and Vector Representations

#### A. What embeddings are?

Embeddings are numeric vectors that represent objects such as words, sentences, images, audio sounds, or diverse scientific data, mapping them into a high-dimensional vector space so that similar items are located close together. In text, for example, words with similar meanings are represented by vectors that are near to each other, while in scientific datasets, experiments or measurements with similar characteristics are encoded in the same way.

These embeddings are generated by machine learning models that learn patterns and semantic relationships within the data. The resulting vectors allow similarity comparisons using different distance metrics such as cosine similarity or Euclidean distance, which are essential for efficient retrieval in vector databases. *(Pan, Wang & Li, 2023 - Section 2; Raja & Vats, 2025 - Sections 2 & 3; Alfredo, 2025 - Section 3)*

Unlike traditional keyword-based search, which matches only exact words, semantic search uses embeddings to find items that are similar in meaning, enabling the retrieval only of the relevant results, even when queries and data do not share the same terms. This capability supports lots of modern applications from nowadays, where finding semantically similar items quick and accurate is very important. *(Pan, Wang & Li, 2023 - Section 2; Raja & Vats, 2025 - Section 4; Alfredo, 2025 - Section 3)*

#### B. How do embeddings work?

Embeddings are generated by machine learning models that process data and learn patterns and relationships. These models convert objects, like words, sentences, images, or different scientific data, into high-dimensional vectors. Items that are similar in meaning or features are placed close together in the vector space.

After the embeddings are created, similarity between items can be measured using distance metrics, like cosine similarity or even Euclidean distance. These metrics provide a numerical way to assess how close two items are based on their characteristics, allowing systems to identify related objects even when the data does not share the same exact words or attributes. *(Pan, Wang & Li, 2023 - section 2; Raja & Vats, 2025 - section 4; Alfredo, 2025 - section 3).*

To support efficient search in large datasets, embeddings are stored and indexed in vector databases. Indexing methods enable fast retrieval of vectors that are closest to a query, making similarity search rapid and scalable. *(Pan, Wang & Li, 2023 - sections 2 & 3; Raja & Vats, 2025 - sections 3 & 4; Alfredo, 2025 - section 3).*

In conclusion, embeddings work by transforming raw data into vectors, using distance metrics to measure similarity and storing these vectors in indexed vector databases to allow efficient retrieval of the related items with the query.

### C. The role of embeddings in Semantic Search

Embeddings are the most important mechanism that enables semantic search, because they transform both user queries and stored data into vectors using the same vector space. This allows the search systems to compare meanings directly, rather than relying on exact keyword overlap. As a result, items that are semantically related can be identified as semantically related even if they do not share the same words. *(Pan, Wang & Li, 2023 – section 2)*

Traditional keyword-based search often struggles when the same idea is expressed using different words. Semantic search beats this limitation by using embeddings to represent not only the data, but also the meaning of it: vector databases store these embeddings and use similarity search to retrieve results that are related in meaning, not just those that contain the same keywords. *(Raja & Vats, 2025 – section 4)* In practice, embeddings enable semantic search across multiple application domains. For example, search engines use embeddings to match queries with documents that are conceptually relevant. Recommendation systems rely on embeddings to suggest the users items similar to their preferences, while AI assistants use embeddings to retrieve contextually close information from large knowledge bases. *(Pan, Wang & Li, 2023 – section 2; Alfredo, 2025 – section 3)*

Overall, embeddings act as the link between raw unstructured data and efficient semantic retrieval. Without embeddings, vector databases would not be able to support scalable semantic search. In their absence, modern AI-driven applications and tools would become totally impractical.

## IV. From Keyword Search to Semantic Search

### A. Keyword Search Limitations

Vector databases confront challenges that traditional keyword-based or relational databases cannot handle, such as high-dimensional data storage, approximate nearest-neighbor search and fast semantic retrieval.

Traditional keyword-based search relies on exact or partial matches between query terms and stored data. While this approach works well for simple queries based on exact word matching, it fails to capture semantic meaning, context, or relationships between concepts. As a result, relevant information may be missed when different words are used to express the same idea or when contextual understanding is required. *(Pan, Wang & Li, 2023 – section 2; Raja & Vats, 2025 – section 4)*

### B. Semantic Search Concept

Semantic search is an information retrieval approach that focuses on understanding the meaning and the intention behind a user query, rather than matching exact keywords. Instead of treating text as a collection of independent letters from the alphabet, semantic search represents both queries and data as embeddings, as numerical vectors that capture semantic relationships. These

embeddings are placed in a shared vector space, where similarity between items is measured using different distance metrics. *(Pan, Wang & Li, 2023 – section 2)*

By operating in a vector space, semantic search could give relevant results even when the query and the stored data use different words. This makes it possible to capture even synonymy, contextual similarity and different conceptual relationships that traditional keyword-based methods cannot handle. Semantic retrieval can handle them, because it uses embeddings and similarity search to move beyond exact keyword matching and basic word search, allowing systems to find relevant results based on meaning rather than identical terms. *(Raja & Vats, 2025 – section 4)*

Semantic search also supports more complex retrieval scenarios, such as searching across large unstructured datasets, where it is handling ambiguous queries and integrating multiple data modalities. In this context, vector databases play a central role by storing embeddings and enabling efficient approximate nearest-neighbor search (ANN). This allows semantic search systems to scale to millions or even to billions of vectors while maintaining a really acceptable query performance. *(Pan, Wang & Li, 2023 – section 3; Alfredo, 2025 – section 3)*

Representing a shift from surface-level text matching to meaning-based retrieval, semantic search is nowadays essential for modern applications.

### **C. Benefits / Transition**

The transition from keyword-based to semantic search enables more flexible, accurate and context-aware information retrieval. Semantic search improves the result's relevance, supports personalization (also the user experience in application contexts) and scales better for large and unstructured datasets.

## **V. Indexing Techniques**

### **A. Purpose**

The main purpose of indexing techniques in vector databases is to enable fast and scalable similarity search over high-dimensional data. As embeddings generated by machine learning models often contain hundreds or sometimes even thousands of dimensions, directly comparing a query vector with all of the stored vectors becomes impractical when it comes to large datasets. Indexing addresses this challenge by organizing vectors in a way that significantly reduces the number of comparisons that are required during a search based on a query. *(Pan, Wang & Li, 2023 – Section 3)*

Another important purpose of vector indexing is to balance efficiency and accuracy. Exact nearest-neighbor search guarantees precise results but it is computationally expensive. In contrast, vector databases prioritize the response time and scalability, especially in real-world applications such as recommendation systems or AI assistants. Indexing techniques make it possible to return results that are sufficiently accurate while also keeping a great performance. *(Raja & Vats, 2025 – Section 5)*

Overall, indexing techniques are essential for transforming semantic search from a theoretical concept into a practical solution that can operate on large-scale and datasets that are in continuous growth.

## B. Methods

Vector databases have a variety of indexing methods to efficiently support similarity search when it comes to high-dimensional embeddings. As embeddings represent semantic information in numerical form and they often have hundreds of dimensions, searching them efficiently requires dedicated approaches that reduce computational cost while keeping the relevance for semantics. So, these methods are designed to reduce computational complexity while maintaining acceptable retrieval accuracy. (*Pan, Wang & Li, 2023 – Section 3*)

The most common indexing methods can be grouped into ***distance-based***, ***partition-based***, ***graph-based***, and ***compression-based*** approaches.

### I. Distance-based Approaches

Distance-based approaches represent the most fundamental method for similarity search in vector databases and serve as the conceptual baseline for all other indexing techniques. These approaches rely directly on computing a distance or similarity measure between vectors in a high-dimensional space. The basic idea is that vectors that are closer to each other (according to a chosen metric) are considered more similar in terms of semantic meaning. (*Pan, Wang & Li, 2023 – Section 3*)

In distance-based methods, no advanced indexing structure is used to reduce the search space. Instead, similarity is determined purely and only by numerical comparison. Although this makes the approach easy to understand and implement for everyone, it also introduces significant scalability limitations when it comes to large datasets.

#### *Distance Metrics and Similarity Measurement*

Distance-based approaches depend the most on the choice of the distance or similarity metric. The most used metrics include Euclidean distance, cosine similarity and inner product. Each of these metrics measures similarity in a slightly different way, but they all aim to capture how close two vectors are in the embedding space.

For example, Euclidean distance measures the straight-line distance between vectors and it is suitable in scenarios where the absolute values and lengths of vectors carry meaningful information. On the other hand, Cosine similarity focuses on the angle between vectors rather than their absolute values, making it particularly suitable for text embeddings.

### Examples of distance/similarity metrics:

**Definition 1 (Hamming)**  $d(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n \delta_{a_i, b_i}$

where  $\delta$  is the Kronecker delta.

Other similarity scores start from an inner product. For  $\mathbb{R}^D$  space, the dot product typically serves as the inner product.

**Definition 2 (Inner Product)**  $f(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n a_i b_i$ ,

also written  $f(\mathbf{a}, \mathbf{b}) = \langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a} \cdot \mathbf{b}$ . The quantity  $\sqrt{\langle \mathbf{a}, \mathbf{a} \rangle}$  defines the Euclidean norm  $\|\mathbf{a}\|$  or magnitude of  $\mathbf{a}$ . Euclidean distance is the norm of the vector resulting from the subtraction of two vectors,  $\|\mathbf{a} - \mathbf{b}\|$ . Euclidean distance is metric.

**Definition 3 (Cosine Similarity)**  $f(\mathbf{a}, \mathbf{b}) = \langle \hat{\mathbf{a}}, \hat{\mathbf{b}} \rangle$

or  $f(\mathbf{a}, \mathbf{b}) = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\|\mathbf{a}\| \|\mathbf{b}\|}$ . Cosine similarity yields the angle between  $\mathbf{a}$  and  $\mathbf{b}$ .

Arbitrary  $p$ -norms,  $\|\mathbf{x}\|_p = (|x_1|^p \dots |x_D|^p)^{1/p}$ , induce the Minkowski distance, generalizing Euclidean distance.

**Definition 4 (Minkowski)** The  $p$ -order Minkowski distance is  $d(\mathbf{a}, \mathbf{b}) = (\sum_{i=1}^n |a_i - b_i|^p)^{1/p}$ ,

or  $d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_p$ . All positive and integer values of  $p$  yield metrics.

**Definition 5 (Mahalanobis)** For any positive semi-definite matrix  $M$ ,  $d(\mathbf{a}, \mathbf{b}) = \sqrt{(\mathbf{a} - \mathbf{b})^\top M (\mathbf{a} - \mathbf{b})}$ .

(Pan, Wang & Li, 2023 – Section 3)

### Brute-Force Nearest-Neighbor Search

The most basic and common distance-based method is brute-force nearest-neighbor search, also known as linear scan. In this approach, the query vector is compared with every vector stored in the database. The distance between the query and each stored vector is computed, and the vectors with the smallest distances are returned as the most similar results.

This method guarantees exact results because no candidates are excluded during the search process, but its computational cost increases linearly with the number of stored vectors. As the dataset grows, query latency increases dramatically, making brute-force search unsuitable for large-scale applications. This is why while brute-force search is accurate, it is not practical for real-world systems that require fast responses. (Pan, Wang & Li, 2023 – Section 3)

### Role of Distance-Based Search as a Baseline

Despite it is inefficient, distance-based brute-force search plays an important role in vector database research and system design. It is usually used as a baseline to evaluate the accuracy of more advanced indexing methods. By comparing approximate search results with exact brute-force results, system designers can measure how much accuracy is sacrificed in exchange for performance improvements. Even so, sometimes distance-based approaches may still be useful, when it comes to small-scale scenarios, in applications where datasets are limited in size (small dataset). (Pan, Wang & Li, 2023 – Section 3)

### Scalability Challenges

The main limitation of distance-based approaches is represented by their poor scalability. As the dataset grows in size and vectors become more



high-dimensional, computing distances for all elements becomes very expensive. This phenomenon is often described as the *curse of dimensionality*, where distance calculations become less effective in high-dimensional spaces. (Pan, Wang & Li, 2023 – Section 3)

In addition, distance-based search does not use any natural organization of the data. Each vector is processed separately, without organizing the data to limit unnecessary comparisons. Because of this, more advanced indexing methods are required to improve efficiency and performance.

### Relationship to Approximate Nearest-Neighbor Methods

Distance-based approaches form the foundation for approximate nearest-neighbor (ANN) methods. Even though ANN tries to cut down on the number of distance calculations, it still uses the same metrics to see how similar the vectors are. Basically, distance-based search provides the exact results that ANN methods aim to approximate, but ANN achieves similar results more efficiently. This is why understanding how distance-based methods work is important, because it is important to see why we need partition-based, graph-based, or compression-based approaches and how they improve, starting from the basic brute-force method and going so on. (Pan, Wang & Li, 2023 – Section 3)

## **II. Partition-based Approaches**

Partition-based indexing is a method that helps vector databases manage large collections of embeddings more efficiently. The main idea is to divide the entire vector space into smaller regions, called partitions or clusters, so that similar vectors end up in the same group. This way, when a query is made, the system only needs to search in the most relevant partitions instead of checking every vector in the database. (Pan, Wang & Li, 2023 – Section 3)

### How Partitioning Works

The process usually starts with clustering. The database analyzes the vectors and identifies representative points, often called cluster centers. Each vector is then assigned to the cluster center it is closest to, based on a distance metric like Euclidean distance or cosine similarity. This creates partitions where vectors that are semantically similar are grouped together. After that, when a query comes in, the system first compares the query vector with the cluster centers to find which partitions are most likely to contain relevant results. Then, it only searches vectors inside those partitions. This approach reduces computation and speeds up queries significantly, especially in large datasets. (Pan, Wang & Li, 2023 – Section 3)

### Candidate Selection and Trade-offs

The number of partitions selected for search affects both speed and accuracy, because looking at more partitions increases the chance of finding all relevant results but also takes more time, while checking fewer partitions makes the search faster but might miss some matches. This is why partition-based

indexing allows system designers to adjust this trade-off depending on the application's needs. *(Pan, Wang & Li, 2023 – Section 3)*

### Scalability and Challenges

Partitioning makes the search scalable. It also works well with other indexing methods, often acting as a first filtering layer before applying graph-based or compression-based techniques. Partitioning is conceptually simple and easy to implement, which makes it useful for any use.

Even so, partition-based indexing depends on how well the clustering reflects the actual distribution of vectors. Poor clustering can cause vectors that are similar to end up in different partitions, meaning the search might miss relevant results. Another challenge is represented by the dynamic type of data: adding or updating vectors frequently may require re-clustering, which can cost a lot and be complicated to manage over time. *(Pan, Wang & Li, 2023 – Section 3)*

### Role in Modern Vector Databases

Modern vector database systems rarely use partitioning alone. Instead, they combine it with other methods to balance speed and accuracy. For example, partitioning can narrow down the search space, after which a graph-based method can navigate local neighborhoods, or compressed vectors can be used for faster distance computations. This layered approach helps to maintain high performance even when it comes to massive datasets. *(Pan, Wang & Li, 2023 – Section 3)*

## **III. Graph-based Approaches**

Graph-based indexing is a method that organizes vectors into a network, or graph, where each vector is a node and edges connect vectors that are close to each other in the embedding space. *(Pan, Wang & Li, 2023 – Section 3)*

The main idea when it comes to graph-based approaches when indexing vectors is that similar vectors tend to form local neighborhoods, and by navigating these neighborhoods efficiently, the system can find nearest neighbors without comparing the query to all of the existing vectors.

### How Graph-Based Indexing Works

In graph-based methods, vectors are connected based on their similarity. Each node stores links to its closest neighbors, creating a structure where searching can move from one node to another one along their edges that lead closer to the query vector. *(Pan, Wang & Li, 2023 – Section 3)*

So, when a query comes in, the search begins at one or more starting points in the graph. The algorithm then hops from node to node, moving closer and closer to vectors that are closer to the query. In this way, the system only looks at the parts of the space that are most likely to have good matches with the user's query, instead of checking every vector in the database.

### Hierarchical Graph Structures

To make searches even faster, many graph-based systems use a layered or hierarchical structure. The top layers give an overview of the data, linking clusters of vectors that are far apart. The lower layers have more detailed connections, letting the system to navigate precisely among closely related vectors. (Pan, Wang & Li, 2023 – Section 3)

So, when a search is performed, the query starts at the top layer to quickly get to the general area where the relevant vectors are. Then it moves down through the layers to do a more detailed search in the most-matched neighborhood. This multi-layer approach helps the system stay both fast and accurate.

### Scalability and Challenges

Graph-based methods offer several benefits, such as efficiency, by following edges through local neighborhoods so the system can avoid scanning all of the existing vectors. Also, this indexing method is used for its accuracy and scalability. Because the graph keeps similar items close to each other, the most relevant vectors are likely to be reached even in very large datasets, very efficiently, especially when combined with hierarchical layers. (Pan, Wang & Li, 2023 – Section 3)

Even so, graph-based indexing has its challenges. Building the graph can be computationally expensive, especially when it comes to massive datasets, and adding or removing vectors requires updating edges, which can be complicated to maintain efficiently over time. Moreover, the number of neighbors per node and the choice of the entry points can influence a lot the search performance and the result quality. (Pan, Wang & Li, 2023 – Section 3)

### Role in Modern Vector Databases

In practice, graph-based indexing is often combined with other methods. For example, partition-based indexing may be used first to narrow down the search space, as previously said, and then graph-based navigation can find the nearest neighbors from the selected partitions. Compression techniques may also be applied to reduce memory usage during graph traversal. (Pan, Wang & Li, 2023 – Section 3) Using this layered or combined approach, vector databases can manage millions of vectors quickly while still finding the most relevant results.

## **IV. Compression-based Approaches**

Compression-based indexing methods focus on reducing the size of the vectors stored in a database so that searches can be done faster and with less memory. Instead of storing the full high-dimensional vectors, these methods represent them in a smaller or in an approximate form. This makes it possible to handle very large datasets without slowing down the system, because compression keeps the most meaningful information in the vectors and it discards the redundant or less important details. This way, similarity search can still find

semantically relevant results without having to process the full original vectors. (Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4)

### How Compression Works

There are several ways to compress vectors. One common method is **quantization**, where vector values are rounded or mapped to a smaller set of numbers. For example, instead of storing a floating-point number with full precision, it might be represented with fewer bits.

Another method is **dimensionality reduction**, which reduces the number of dimensions in a vector while keeping most of the important information.

So, when a query comes in, the system compares the compressed query vector to the compressed vectors in the database. This reduces the number of calculations and the amount of memory which is needed, allowing searches to be faster without losing too much accuracy. (Pan, Wang & Li, 2023 – Section 3)

### Scalability and Challenges

Compressed vectors take less space, so more vectors can fit in memory. Also, comparing smaller vectors requires fewer calculations, which speeds up the queries. Moreover, compression allows the system to handle millions or even billions of vectors efficiently. (Pan, Wang & Li, 2023 – Section 3) It is often combined with other methods like partitioning or graph-based approaches, and this combination keeps the search both fast and accurate.

On the other hand, while compression makes the searches faster, it comes with some trade-offs. Compressed vectors are only an approximation, so results may be slightly less accurate. Also, compressing and decompressing vectors requires some additional computation, and the level of compression needs to be carefully chosen to balance speed and accuracy. (Pan, Wang & Li, 2023 – Section 3)

Vector databases use different indexing methods to make similarity search faster and more efficient.

Distance-based methods are the simplest: the query is compared to every vector in the database. This guarantees exact results, but the approach becomes very slow as the dataset grows.

Partition-based methods improve efficiency by dividing the vector space into clusters or partitions. When a search is performed, only the most relevant partitions are explored, which reduces computation while still finding most similar items.

Graph-based methods take a different approach by connecting vectors to their closest neighbors in a graph. The search moves along these connections, exploring only promising areas, and hierarchical layers can further speed up the process. (Pan, Wang & Li, 2023 – Section 3)

Compression-based methods reduce the size of vectors, sometimes using semantic compression, which saves memory and speeds up the search while also maintaining a great result quality. (Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4)

### C. Advanced Optimizations

Modern vector databases do not just rely on basic indexing. They use advanced techniques to make searches faster and more accurate. For example, combining partition-based, graph-based, and compression-based methods, databases can focus the search on the most relevant vectors, they can move efficiently through neighborhoods and reduce memory usage at the same time. *(Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4)*

## VI. Modern Vector Database Systems

### A. About Modern Vector Database Systems

Modern vector database systems are designed to handle the massive amounts of high-dimensional data generated by AI models, including embeddings from large language models (LLMs). Unlike traditional relational databases, which are optimized for structured data, vector databases are built to store, index, and search vectors efficiently, even when the data is unstructured, such as text, images, audio, or different multimodal combinations. *(Pan, Wang & Li, 2023 – Section 3)*

These systems rely on a combination of indexing methods to ensure that searches remain fast and scalable. As previously presented, while distance-based search provides exact results, it is too slow for large datasets. Partition-based indexing organizes vectors into clusters, allowing the system to focus only on the most relevant regions. Graph-based indexing connects vectors to their nearest neighbors, enabling searches to visit only promising areas. Compression reduces memory use and accelerates computation, sometimes using semantic compression to preserve the most meaningful information. *(Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4)*

Most modern vector databases have three main parts. The storage layer keeps the raw vectors and their metadata. The indexing layer organizes the vectors so that searches are fast. And the query layer handles incoming searches, finding and returning the most relevant results efficiently. *(Pan, Wang & Li, 2023 – Section 3)*

These modern vector database systems are a key component of AI-driven applications. By combining multiple indexing methods, they can handle huge amounts of high-dimensional data efficiently and provide fast, accurate similarity search. These systems make it possible to work with unstructured and multimodal data, supporting everything from semantic search and recommendation systems to AI assistants and scientific analysis. The hybrid design and advanced optimizations ensure that vector databases can scale to millions of vectors while maintaining high retrieval quality, making them an essential tool for modern data-driven technologies. *(Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4)*

## B. Architecture / Features

Modern vector databases usually combine different indexing methods in a hybrid setup. For example, a search might first use partitioning to skip unrelated vectors, then move through a graph to find the closest matches, and finally use compressed vectors to quickly calculate similarities. This layered approach keeps searches fast, uses memory efficiently, and still finds accurate results, even when dealing with millions of vectors.

Another useful feature is that many vector databases can update in real-time or almost real-time. This means new embeddings from streaming data or AI-generated content can be added without rebuilding the whole index. Many systems also use distributed storage and parallel processing, so they can handle datasets that are too big for only one machine. *(Raja & Vats, 2025 – Section 4)*

Vector databases are often used together with AI models, especially large language models (LLMs), in systems like retrieval-augmented generation (RAG). In this setup, embeddings from documents or knowledge bases are stored in the database, and when the AI gets a query, it can quickly fetch the most relevant embeddings to give better and more accurate answers. *(Pan, Wang & Li, 2023 – Section 3)*

Besides text, many systems support multimodal embeddings, and this means that vectors from text, images, or audio can all exist in the same space in the same time. This makes it possible, for example, to search for an image using a text description or provide recommendations that consider different types of data at once. *(Alfredo, 2025 – Section 4)*

In short, vector databases aren't just for storing data. They are a key part of AI applications, helping with semantic search, recommendations, and smart retrieval in ways that traditional databases can not. Their combination of hybrid indexing, scalability, multimodal support, and AI integration makes them essential for handling nowadays large and complex datasets. *(Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4; Alfredo, 2025 – Section 4)*

## C. Examples / Applications

Vector databases are used in many different areas where understanding the meaning of data is important. One of the most familiar examples is search engines. Unlike traditional keyword-based search, which only looks for exact word matches, search engines powered by vector databases can understand what a user is really asking. For instance, if someone searches for "the best horror movies of the 21st century," the system can find movies that belong to the horror genre, even if the exact phrase isn't in the movie titles or descriptions. This is possible because embeddings capture the meaning behind words, and vector databases efficiently store and search these embeddings. *(Pan, Wang & Li, 2023 – Section 3)*

Another area where vector databases shine and have an important meaning is recommendation systems. These systems are everywhere, even if we talk about e-commerce, streaming services or social media platforms. Vector databases store embeddings representing items, like products, movies, or posts, alongside embeddings representing users's preferences. By comparing vectors, the system can suggest items that are similar in style or meaning to what the

user likes the most. For example, if a user enjoys a particular type of movie, the system might suggest other films with similar plots or themes, even if the user hasn't watched them before. This approach allows recommendations to feel more personalized and accurate. *(Raja & Vats, 2025 – Section 4)*

AI assistants and chatbots also rely heavily on vector databases. They store embeddings from large knowledge bases, which allows the system to find relevant answers even if the user's question is phrased differently from the stored information. For example, if someone asks, "How do I connect my phone to my TV?" the system can find instructions in a manual or anywhere else, even if the wording is slightly different. This makes AI assistants more flexible and able to understand context and the user's intent. *(Pan, Wang & Li, 2023 – Section 3)*

Vector databases are extremely useful in scientific research as well. Large datasets in fields like genomics, astronomy, or climate science often contain millions of complex, high-dimensional data points. Using vector databases, researchers can perform similarity searches to find patterns, compare experiments, or identify data points that are closely related. Without vector databases, querying such large and complex datasets would take an impractical amount of time. *(Alfredo, 2025 – Section 3)*

Looking at specific systems, there are several notable examples.

- **Milvus** is designed for massive-scale deployments and supports hybrid indexing methods, including graph-based and partition-based indexing. This allows it to handle millions of vectors efficiently, so it is ideal for large enterprise applications.
- **Weaviate** combines vector search with semantic APIs and knowledge graph features, which is useful when relationships between data points matter as much as the data itself.
- **Pinecone** is built for cloud-based applications, offering real-time updates and automatic scaling, which makes it perfect for AI-powered recommendations or chatbots that require low latency.
- **Qdrant** focuses on performance and has rich filtering options.
- **Chroma** is often used for smaller projects or experiments, where developers want fast and easy-to-use vector search functionality.

*(Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4)*

It's also important to mention that many traditional databases now offer vector search features. Systems such as Redis, Elasticsearch and OpenSearch can store vector fields and perform approximate nearest-neighbor searches. This is useful for applications where you want both structured queries (like filtering by category for example) and semantic search at the same time. For example, in an online store, you might want to filter products by price or brand while also finding items that are similar to ones the user liked. *(Raja & Vats, 2025 – Section 4)*

Multimodal search is another area where vector databases excel. Some modern systems allow text, images, and audio to exist in the same vector space at the same time. This means you can search for an image using a description,

find a video using an audio sample, or create recommendations that combine multiple types of data. By supporting different data types together, vector databases allow much richer and more flexible search experiences than traditional systems. (Alfredo, 2025 – Section 4)

Beyond these, vector databases are also a key for real-time AI applications. In retrieval-augmented generation (RAG), embeddings from documents or knowledge bases are stored in the database. When an AI model receives a query, it can instantly fetch the most relevant vectors, improving the relevance and accuracy of the output. This makes them essential in systems where quick responses and meaningful answers are required. (Pan, Wang & Li, 2023 – Section 3)

In conclusion, vector databases are not just about storing vectors, because they are a foundation for modern AI-powered systems. They make semantic search, recommendations, and intelligent retrieval possible in different industries. Their combination of hybrid indexing, real-time updates, distributed processing, multimodal support and AI integration allows them to handle huge, complex datasets efficiently. The growth of these systems shows how essential they have become for modern applications from nowadays. (Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4; Alfredo, 2025 – Section 3)

## VII. Advantages and Limitations

### A. Advantages

Vector databases have several strong advantages that make them stand out from traditional databases. One of their biggest advantages is represented by their ability to **perform semantic search**. Traditional search systems only match exact words, which means they can miss items that are related in meaning but don't contain the exact words from the given query. Vector databases, on the other hand, store embeddings, which allows the database to find items that are similar in meaning, not just in wording. This capability is what powers intelligent search engines, recommendation systems and AI assistants, making them much more useful and user-friendly. (Pan, Wang & Li, 2023 – Section 3)

Another important advantage of vector databases is represented by their **speed and scalability**. Modern vector databases can handle millions or even billions of vectors efficiently. Their representative techniques like partitioning, graph-based search, and compression help the system to reduce the number of comparisons it has to make, which means queries return results quickly even when it comes to having huge datasets to deal with. Systems can also be distributed across multiple servers, allowing them to process many queries at the same time without slowing down the whole process. This is particularly important for applications like e-commerce, streaming services or large-scale research datasets where quick responses are essential. (Raja & Vats, 2025 – Section 4)

Vector databases are also **flexible**. Many modern systems support **multimodal embeddings**, meaning vectors from different types of data can coexist in the same space. This flexibility opens up new possibilities for AI-powered applications that require understanding of diverse data types. **Real-time updates** are another advantage, considering that many systems let you add new embeddings or update existing ones without rebuilding the entire



database from the start each time a change is needed. This is crucial for dynamic environments, like news platforms, social media or online shopping, where data changes constantly so it should always be up to date. *(Pan, Wang & Li, 2023 – Section 3)*

Finally, vector databases integrate smoothly with **AI models**, especially large language models (LLMs), which enable AI systems to give contextually relevant answers, generate real-time recommendations, or retrieve information in a meaningful way.

## B. Limitations

Even with all these advantages, vector databases also have some limitations that need to be considered. One major challenge is the **complexity**. Setting up a system that combines hybrid indexing, distributed storage, and real-time updates can be complicated. Developers need to plan carefully to avoid different inconsistencies in search results, which can be difficult in large-scale deployments.

Another limitation is represented by the **resource usage**. High-dimensional vectors require significant memory and processing power, especially when searching in very large datasets. This means running a vector database at scale can be expensive in terms of hardware or cloud services. Additionally, some systems require GPUs or other specialized hardware to achieve fast query times, which can add to costs. *(Raja & Vats, 2025 – Section 4)*

Vector databases also involve a **trade-off between accuracy and speed**. Many systems use approximate nearest neighbor (ANN) methods to make searches faster. While these methods are usually accurate enough for most applications, they don't always find the exact closest vectors. In some cases, this can be an issue for applications that need perfect precision, such as scientific research or medical data analysis. **Maintenance** is another practical limitation, considering that vector databases often need monitoring, occasionally even reindexing the vectors and updates to ensure optimal performance.

Last but not least, the **data quality** is crucial. Poorly generated or inconsistent embeddings can lead to irrelevant results that don't help at all. Ensuring that the AI models produce reliable embeddings and that the data is clean and well-prepared is essential for vector databases to work effectively efficient and correct.

## C. Practical Considerations

When using vector databases, there are several practical points to keep in mind. First, it's important to understand the **type of data** you have. Different types of data need different ways of generating embeddings, and not every vector database handles multimodal embeddings in the same way. Choosing a system that matches your data and application needs is essential for the start.

**Scaling** is another key consideration. If you expect to store millions or billions of vectors, you need a database that can scale horizontally, distributing both data and queries across multiple machines. Many systems also support parallel query processing, which allows multiple searches to happen at the same time without affecting performance. Also, choosing the **right indexing method** is important, because brute-force distance calculations are accurate but slow,

while partition-based, graph-based, and compression methods speed up searches but may slightly reduce accuracy. Picking the right combination depends, of course, on users's priorities.

Integration with **AI pipelines** is another practical concern. Vector databases are often used together with LLMs or other AI models. Planning how the embeddings will flow between the AI model and the database is essential for a smooth system. **Cost and infrastructure** should also be considered, while running large-scale vector databases requires memory, storage, and lots of processing resources.

## VIII. Recent Trends and Future Directions

Vector databases are still a relatively new technology, but they are evolving very fast. As applications grow larger and data becomes more complex, researchers and engineers constantly look for better ways to make vector search faster, cheaper and more accurate - they want to bring it in the best form.

In recent years, several important trends have appeared, especially around hybrid search, compression, graph-based techniques, and integration with AI models. These trends show how vector databases are moving from simple similarity search tools to full components of modern AI systems.

### A. Trends

In recent years, vector databases have changed a lot compared to their early versions. At the beginning, most systems focused mainly on simple nearest neighbor search, usually using one indexing technique and basic distance calculations. As datasets became larger and AI applications more complex, these simple approaches were no longer enough. This led to several trends that now define how modern vector database systems are built and used.

The most important trends in vector databases for semantic search include hybrid search, semantic compression, graph-augmented retrieval, and multimodal embeddings.

#### Hybrid Search

Hybrid search is one of the most practical and adopted trends in vector databases today. The basic idea behind hybrid search is that relying on a single technique is usually not efficient enough when working with large, real-world datasets. Each indexing or search method has strengths and weaknesses, so combining them allows systems to perform better overall.

In a hybrid search setup, a query is often processed in multiple stages. First, a fast method is used to reduce the search space. For example, partition-based techniques can group vectors into clusters, which makes it possible to ignore large parts of the dataset that are clearly unrelated to the query. This first step does not aim to find the exact best match, but only to narrow down the candidates for the given query's response.

After this filtering step, more precise methods are applied. Graph-based search is commonly used at this stage. By navigating through a graph of vectors that are connected to their nearest neighbors, the system can move quickly between vectors that are more similar to the query.

Finally, distance-based calculations are used to calculate and rank the remaining candidates and to return the best results.

Hybrid search is popular because it allows systems to balance different goals. If speed is more important, the system can stop the search earlier or explore fewer nodes. Otherwise, if accuracy is more important for example, it can explore more of the graph or refine the results further. This flexibility makes hybrid search suitable for many applications, from real-time recommendation systems to large-scale semantic search engines. *(Pan, Wang & Li, 2023 – Section 3)*

### Semantic Compression (Demo available too)

One of the main challenges in vector databases is handling very large numbers of high-dimensional vectors efficiently. Embeddings generated by modern AI models often have hundreds or even thousands of dimensions. While these rich in data representations are useful for capturing semantic meaning, they also require significant memory and computational resources. This becomes a serious problem when vector databases need to store and search through millions or even billions of vectors.

The concept of semantic compression is useful in this case by reducing the size of vector representations while still giving as much semantic information as possible. Instead of storing full, high-precision embeddings, vector databases apply compression techniques that make vectors smaller, faster to process and easier to store. The key idea is that not all numerical details of an embedding are equally important for semantic similarity search. Traditional vector search systems often focus only on finding the nearest neighbors using exact or approximate distance calculations. However, as datasets grow larger and larger, efficiency becomes just as important as accuracy. Semantic compression is presented as a way to balance these two aspects by keeping the meaningful parts of embeddings while removing redundancy or less relevant information. *(Raja & Vats, 2025)*

In practice, semantic compression can take multiple forms. One simple approach is **reducing numerical precision**, for example, by storing embeddings as lower-precision floating-point numbers instead of full 32-bit values. Another common method is **dimensionality reduction**, where embeddings are transformed into vectors with fewer dimensions, while they are still preserving the main semantic structure of the data. These techniques do not aim to perfectly reconstruct the original vectors, but rather to keep enough semantic information so that similarity search still produces useful and relevant results. *(Raja & Vats, 2025)*

An important observation to make is that semantic similarity stays the same even if the numbers change a little, which means that even if embeddings are compressed, texts or items that are semantically close before compression usually remain close after compression. Because of this, semantic compression can significantly reduce memory usage and improve search speed without

significantly affecting the quality of the results. This makes it especially useful in large-scale systems where performance and scalability are critical.

Beyond memory savings, semantic compression also plays a role in improving search efficiency. Smaller vectors require fewer computations during similarity search, which leads to faster query responses. When combined with other indexing techniques, such as graph-based or partition-based methods, compression helps vector databases to scale to much larger datasets than usually. So, semantic compression is not meant to replace existing indexing methods, but rather to complement them and make the overall system more efficient.

*Inspired by the ideas presented in the "Beyond Nearest Neighbors" article, I also implemented a small-scale semantic search system using open-source tools. Text data is loaded from a .csv file (AG News dataset from Kaggle). Sentence embeddings are generated using the pretrained all-MiniLM-L6-v2 model from HuggingFace (384 dimensions). Embeddings are then compressed using PCA to lower dimensions (64D and 32D) while preserving semantic information. Semantic search is performed on full embeddings and compressed embeddings for comparison. This experiment aims to show that compressed embeddings can still support meaningful semantic search while reducing vector size and improving efficiency.*

Overall, semantic compression represents an important step in making vector databases more practical for real-world applications. As datasets continue to grow and AI models generate increasingly complex embeddings and by focusing on the meaning rather than exact numerical values, semantic compression enables scalable and efficient semantic search, which is a very important requirement for modern AI-driven systems/applications.

### Graph-Augmented Retrieval

Graph-augmented retrieval is another important trend that has proven to be very effective for similarity search. In this approach, vectors are organized into a graph structure, where each vector is connected to a small number of its nearest neighbors. The main advantage of this structure is that similar vectors tend to be close to each other in the graph. When a query is made, the system does not compare it with all vectors in the database. Instead, it starts from one or more entry points and moves through the graph step by step. At each step, it follows edges from the vectors that are more similar to the query. This allows the search to focus only on promising areas of the vector space, rather than wasting time on distant and irrelevant vectors.

Many modern systems use hierarchical graphs to improve performance even further. Higher levels of the graph provide an overview of the data, connecting distant clusters. Lower levels contain more detailed connections between vectors that are near to each other. During a search, the system starts at a higher level to quickly reach the right region, then moves down to refine the search into a more detailed region, where it can fetch the best results.

In the end, graph-augmented retrieval works well because it takes advantage of the natural structure of vector spaces. It is especially effective in high-dimensional settings, where brute-force distance calculations become

inefficient. This is why graph-based methods are now a crucial component of many production-level vector databases. (*Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 3*)

### Multimodal Embeddings

In recent years, vector databases have expanded their capabilities to handle multimodal data, which means combining different types of information in the same vector space is now possible. Instead of working only with text or only with images, modern systems can store embeddings from multiple data types: text, images, audio, or even combinations of these. This way, they can be compared directly. This is particularly useful in applications where you want to find relationships using different modalities, such as searching for an image based on a text description or recommending music based on written reviews.

The main idea behind multimodal embeddings is to represent different types of data in a shared vector space. This way, vectors that are semantically related, even if they come from different modalities, are close to each other in the space. For example, the *sentence* “a blue sports car” can have a vector that is close to the embedding of an actual *image* of a blue sports car. This allows vector databases to perform similarity search across modalities without needing separate systems for each type of data. (*Raja & Vats, 2025 – Section 4; Pan, Wang & Li, 2023 – Section 3*)

Creating multimodal embeddings usually involves using pretrained AI models for each modality. Text can be transformed into embeddings using language models like Sentence-Transformers. Images can be encoded using convolutional neural networks or vision transformers, while audio can be processed using spectrogram-based models or audio transformers. Once each type of data is transformed into vectors, additional layers or training methods align the embeddings into a common space, ensuring that semantically related items are close to each other. (*Raja & Vats, 2025 – Section 4*)

A major advantage of multimodal embeddings is that they enable bigger search and recommendation experiences. For instance, in an e-commerce platform, a user could take a picture of a product and retrieve similar items from text-based product descriptions. Similarly, a video recommendation system could find related videos based on both audio content and subtitle text. This flexibility is becoming more common in AI-powered applications, where combining multiple data sources improves user experience and relevance. (*Alfredo, 2025 – Section 4*)

Despite their benefits, multimodal embeddings introduce some challenges. Storing multiple types of embeddings in the same vector database can increase memory and computation requirements. Also, aligning vectors from different modalities requires careful training and evaluation to ensure that vectors truly reflect semantic similarity across modalities. (*Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4*)

## **B. Integration with AI / LLMs**

Retrieval-Augmented Generation, known as RAG, is one of the most interesting ways in which vector databases are used with AI today. The basic idea is that instead of letting a language model (LLM) answer a question purely from

its internal knowledge, we first retrieve relevant information from a vector database and then use that information to guide the model's answer. This allows AI systems to produce answers that are much more accurate and context-aware than usual. (*Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4*)

Basically, when a user gives a query, the system first converts the question into a vector embedding, just like the documents or knowledge items stored in the database. The vector database then performs a similarity search and retrieves the vectors that are closest to the query in terms of meaning, not just exact words. These retrieved embeddings are passed to the LLM, which uses them as additional context while generating its response. The result is a system that can answer questions about very recent events, specific facts, or specialized domains that the model alone might not know. (*Alfredo, 2025 – Section 4*)

This setup has multiple advantages. First, it improves accuracy, because the model bases its answers on actual data rather than just patterns learned during training. Second, it allows AI systems to handle dynamic knowledge bases, since new embeddings can be added without retraining the model again and again. Third, it supports context-aware answers, which is essential for applications like customer support, research assistance or even educational tools. (*Raja & Vats, 2025 – Section 4*) It is also fast, because the LLM doesn't have to guess or scan everything; it only works with the most relevant vectors, which saves time and computational resources.

An interesting aspect is multimodal RAG. If the vector database supports multimodal embeddings (text, images, audio), the system can retrieve and combine information across different data types too. (*Alfredo, 2025 – Section 4; Raja & Vats, 2025 – Section 4*) This flexibility shows how vector databases extend the capabilities of AI beyond traditional single-modality systems.

To conclude, RAG demonstrates how vector databases and LLMs work together: the database finds relevant embeddings, and the AI uses them to give better, more accurate answers. This combination is becoming essential in AI-powered applications, from research assistants and chatbots to recommendation engines and knowledge management tools. (*Pan, Wang & Li, 2023 – Section 3; Raja & Vats, 2025 – Section 4; Alfredo, 2025 – Section 4*)

### **C. Future Research / Opportunities**

While vector databases are already powerful, there are still many areas for improvement and research. One of the main opportunities is developing self-updating indexes. As new data is constantly generated, vector databases need to add new embeddings quickly without rebuilding the entire index. Incremental indexing methods are being explored to make updates faster while keeping the search accuracy high. (*Pan, Wang & Li, 2023 – Section 3*)

Hardware acceleration is another promising area. High-dimensional vector searches can be computationally expensive, especially when dealing with millions or billions of vectors. Using GPUs, TPUs, or specialized hardware can speed up searches dramatically. Future systems may also combine multiple hardware types or use distributed processing to handle extremely large datasets efficiently. (*Raja & Vats, 2025 – Section 4*)

Scaling strategies are crucial for real-world applications. Systems must scale horizontally to handle massive numbers of embeddings. Techniques like

sharding vectors across servers, distributed graph search, and hybrid partitioning are being studied to allow databases to remain fast even at huge scales. This is important for AI applications that need real-time responses from large knowledge bases. *(Raja & Vats, 2025 – Section 4)*

Another interesting future direction when it comes to vector databases for semantic search is the adaptive compression and hybrid search. Compression reduces memory usage and speeds up computation, but future methods may adjust compression dynamically based on usage or query patterns. Hybrid search combines different indexing techniques to balance speed, memory, and accuracy for specific applications. *(Raja & Vats, 2025 – Section 4; Pan, Wang & Li, 2023 – Section 3)*

Finally, all these research areas connect directly to AI integration. Improvements in updating indexes, scaling, and hardware acceleration helps the RAG pipelines, multimodal search systems, and recommendation engines to be more powerful. The goal is to make vector databases smarter, faster, and more adaptable, allowing AI systems to handle increasingly complex and large datasets. *(Alfredo, 2025 – Section 4)*

## IX. Conclusion

This paper explored the vector databases for semantic search, showing how these systems have changed the way we store, retrieve, and use unstructured data. We started by understanding why keyword search is no longer enough, and after that we learned how vector databases work. We looked at how embeddings work, how they allow semantic search, and how vector databases use them to find similarities and make recommendations. Indexing techniques were explained in detail, showing how modern systems balance accuracy, speed, and memory usage.

Modern vector databases are more than just storage, because they are integrated into AI pipelines, especially when working with large language models (LLMs). With this topic on, we also discussed retrieval-augmented generation (RAG). The report also looked at trends and advanced techniques, such as semantic compression, graph-augmented retrieval, multimodal embeddings and hybrid search. These innovations help vector databases work faster, use less memory, and support multiple types of data, including text, images, and audio. We saw that compression can reduce storage needs while keeping results as accurate as possible, and we also saw that multimodal embeddings allow the system to link information from different sources at the same time.

Finally, we considered future directions for vector databases. Research is focusing on self-updating indexes, hardware acceleration, distributed and scalable architectures and adaptive search methods. These improvements will make vector databases even more efficient, intelligent, and integrated with AI applications.

In short, vector databases are becoming essential for modern AI systems, providing the tools needed to handle unstructured data in a meaningful, practical, and scalable way.

## References

- [1] James Jie Pan, Jianguo Wang & Guoliang Li, *Survey of Vector Database Management Systems*, 2023, <https://arxiv.org/pdf/2310.14021>, Last accessed: January 2026
- [2] Cuzzocrea Alfredo, *Vector Databases for Modelling, Managing and Querying Big Scientific Data: Models, Issues, Paradigms*, 2025, <https://dl.acm.org/doi/pdf/10.1145/3733723.3742469>, Last accessed: January 2026
- [3] Rahul Raja & Arpita Vats, *Beyond Nearest Neighbors: Semantic Compression and Graph-Augmented Retrieval for Enhanced Vector Search*, 2025, <https://arxiv.org/pdf/2507.19715>, Last accessed: January 2026
- [4] OpenAI, ChatGPT, <https://chatgpt.com/>, Last accessed: January 2026